

代码是软件应用的文档，执行轨迹是 AI 应用的文档

In software, the code documents the app. In AI, the traces do.

LangChain · Harrison Chase · 2026-01-10 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

要点速览

- 在传统软件中，你通过阅读代码来了解应用的行为——决策逻辑存在于代码库中
- 在 AI Agent 中，代码只是脚手架——真正的决策由模型在运行时做出
- 因此，了解应用行为的可信依据从代码转向了执行轨迹——执行轨迹记录了 Agent 实际做了什么，以及为什么这样做
- 这改变了我们调试、测试、优化、监控、协作以及了解产品使用情况的方式
- 如果你构建 Agent 时没有建立良好的可观测性，就缺少了了解系统实际行为的可信依据

在传统软件中，出了问题，你会读代码。想弄清一个功能如何工作，你会读代码。想提升性能，你会对代码做性能分析。代码就是可信依据。

在 AI Agent 中，这一套不再适用了。

为什么代码无法记录 Agent 的行为

在传统软件中，如果你想了解用户提交表单时会发生什么，只需打开 `handleSubmit()` 并阅读这个函数。决策逻辑就在那里：验证输入、检查身份认证、调用 API、处理错误。它是确定性的——相同的输入、相同的代码执行路径、相同的输出。

在 AI Agent 中，代码只是脚手架。

下面是 Agent 实际代码的一个简化版本：

```
agent = Agent(  
    model="gpt-4",  
    tools=[search_tool, analysis_tool, visualization_tool],  
    system_prompt="You are a helpful data analyst..."  
)  
result = agent.run(user_query)
```

你定义了各个组成部分：使用哪个模型、哪些工具、什么指令。但决策逻辑并不在你的代码中。代码只是负责对大语言模型（LLM）调用进行编排。

真正的决策——何时调用哪个工具、如何通过推理解决问题、何时停止、优先处理什么——这一切都由模型在运行时做出。



随着 LLM 主导应用中越来越多的行为（Agent 就是这种情况），仅靠阅读代码，你能了解的应用实际行为就越来越少。

你仍然可以调试编排代码——工具调用是否正常、解析是否正常。但你无法调试智能本身。Agent 是否做出了好的决策、是否进行了有效的推理——这些逻辑存在于模型中，而不在你的代码库里。

执行轨迹成为新的文档

那么，实际行为记录在哪里？在执行轨迹中。

执行轨迹是 Agent 所采取的一系列步骤。它记录了应用的逻辑——每一步的推理、调用了哪些工具以及为什么调用、执行结果和耗时。



这意味着，在软件世界中针对代码进行的操作，如今到了 Agent 世界中，就要针对执行轨迹来进行。

调试、测试、性能分析、监控——所有这些操作的对象，都从代码转向了执行轨迹。

在传统软件中，如果两次运行产生不同的输出，你会认为输入或代码有所不同。在 AI Agent 中，相同的输入配合相同的代码，也可能产生不同的输出：不同的工具调用、不同的推理链、不同的结果。

要了解发生了什么，唯一的办法就是查看执行轨迹。为什么任务 A 成功了，任务 B 却失败了？比较它们的执行轨迹。对提示词的修改是否改善了推理？比较修改前后的执行轨迹。为什么 Agent 不断犯同样的错误？查看多条执行轨迹中呈现出的规律。

这如何改变 Agent 的构建方式

当了解逻辑的可信依据从代码转移到执行轨迹时，其他一切也会随之改变。过去针对代码进行的所有操作——调试、测试、优化、监控——如今都需要以执行轨迹为中心。我们来看看这在实践中意味着什么。

调试变成执行轨迹分析

当用户反馈“Agent 失败了”时，你不会打开代码寻找缺陷，而是会打开执行轨迹，查找推理在哪里出了问题。Agent 是误解了任务？调用了错误的工具？还是陷入了循环？

这个“缺陷”并不是代码中的逻辑错误，而是 Agent 实际执行过程中出现的推理错误。

举个例子：某个 Agent 对同一个失败的 API 调用反复重试五次，然后才放弃。你的代码中有重试逻辑——它运行得完全正常。缺陷在于，Agent 没有从错误消息中吸取教训。只有查看执行轨迹，你才能看到这一点：相同的工具调用、相同的参数、相同的失败，不断重复。

你无法在推理中设置断点

在传统软件中，发现缺陷后，你会在代码中设置一个断点。

在 AI Agent 中，你无法在推理中设置断点。决策发生在模型内部。

但你可以结合执行轨迹与交互调试环境（playground），在逻辑中设置断点。打开执行轨迹，将它定位到某个特定时刻——恰好在 Agent 做出错误决策之前。把那一刻的完整状态加载到交互调试环境中。交互调试环境就像调试器，只不过调试的对象是推理，而不是代码。

你可以看到：Agent 当时有哪些上下文？它的记忆中有什么？有哪些工具可用？提示词是什么样的？然后开始迭代——调整提示词、修改上下文、尝试不同方法——观察 Agent 是否做出了更好的决策。

测试转向评测驱动

既然了解逻辑的可信依据现在存在于执行轨迹中，你就需要测试这些执行轨迹。这意味着两件事：

第一：你需要一套将执行轨迹加入测试数据集的流程。在 Agent 运行过程中，采集执行轨迹，将其加入可用于评测的数据集。

第二：你需要评测生产环境中的执行轨迹。在传统软件中，你在部署前完成测试，然后发布。在 AI 中，Agent 具有非确定性，因此你需要在生产环境中持续开展评测，以发现质量下降和漂移。

性能优化发生变化

在传统软件中，你对代码做性能分析，找出热点循环并优化算法。在 AI Agent 中，你对执行轨迹做性能分析，找出决策模式——不必要的工具调用、冗余的推理、低效的执行路径。瓶颈在于 Agent 的决策，而这些决策只存在于执行轨迹中。

监控从运行可用性转向质量

一个 Agent 可以处于“正常运行”状态，错误数为 0，表现却仍然糟糕透顶——成功完成了错误的任务、以 10 倍的成本低效地完成了任务，或给出正确却没什么帮助的回答。

你需要监控的是决策质量，而不仅仅是系统健康状况——包括任务成功率、推理质量、工具使用效率。不对执行轨迹进行采样和分析，就无法监控质量。

协作转移到可观测性平台

在传统软件中，协作发生在 GitHub 上。你审查代码、在拉取请求（PR）中留下评论、在 issue 中讨论实现方式。代码是所有人共同围绕的工作产物。

在 AI Agent 中，逻辑并不在代码里，而是在执行轨迹中。因此，协作也必须发生在执行轨迹所在的地方。当然，你仍然会使用 GitHub 来协作开发编排代码。但当你调试 Agent 为什么做出了错误决策时，你需要分享一条执行轨迹，在特定的决策点添加评论，讨论它为什么选择这条路径。你的可观测性平台会成为协作工具，而不仅仅是监控工具。

产品分析与调试融为一体

在传统软件中，产品分析与调试是分开的。Mixpanel 告诉你用户点击了什么，错误日志告诉你哪里出了故障。它们是用于回答不同问题的不同工具。

在 AI Agent 中，两者融为一体。不了解 Agent 的行为，就无法理解用户行为。当你在分析中看到“30% 的用户感到不满”时，你需要打开执行轨迹，看看 Agent 做错了什么。当你看到“用户在请求数据分析功能”时，你需要查看执行轨迹，了解 Agent 已经在选择哪些工具，以及哪些做法行之有效。用户体验就是 Agent 做出的决策，而这些决策记录在执行轨迹中——因此，产品分析必须建立在执行轨迹之上。

做出转变

在传统软件中，代码就是你的文档。在 AI Agent 中，执行轨迹就是你的文档。

这一转变很简单：当决策逻辑从代码库转移到模型中时，你的可信依据也就从代码转移到了执行轨迹。



过去围绕代码进行的一切——调试、测试、优化、监控、协作——如今都要围绕执行轨迹进行。

要做到这一点，你需要良好的可观测性。你需要能够搜索、筛选和比较的结构化执行轨迹；需要能够查看完整的推理链——调用了哪些工具、耗时多久、成本多少；还需要能够对历史数据运行评测，持续监控质量随时间的变化。

如果你正在构建 Agent，却没有这些能力，那就是在盲目摸索。真正重要的逻辑，只存在于这些执行轨迹之中。